tresata

TAKE HOME ASSIGNMENT

SECTION 1 - OVERVIEW

Finance is one of the largest industries in the world and one of the most document-heavy: tax rules, audit evidence, regulations and much of the work is still done by experts reading, checking and cross-referencing by hand. As an engineer walks into that kind of environment, finds where AI can genuinely help, and builds something people trust.

Your job: pick an area of finance, define one simple problem that exists, and design an AI-based solution to it.

Finance is full of problems like these. Following are a few such examples. Please try to avoid using them as part of your own problems.

1. Banks and lenders receive a steady stream of new circulars and rule changes from their regulators. A compliance team has to read each one, work out which internal policies and processes it affects, and prove later that they acted on it. Today that is mostly done by people reading PDFs and updating spreadsheets, and a missed change can mean a fine.
2. Countries sign trade agreements that cut tariffs, but many small exporters never claim the lower rates. The rules on where a product counts as "made" are long and technical, and a small business can't afford a specialist to check every shipment. The agreement exists; the value it was meant to create for the country goes unused.

Pick any area of finance (tax, audit, regulation, cross-border agreements, or something else) and find a problem of your own.

SECTION 2 - YOUR TASK

Work through the steps.

1. **Define one problem.** Define the problem. Explain why this problem matters and why now. State what is out of scope.
2. **Design the solution.** Explain how your proposed solution addresses the problem. Describe what data goes in, what the AI does, what comes out, where human review is needed, and how the system handles uncertainty or mistakes. Explain why AI is necessary instead of a simpler, non-AI approach.
3. **Prove it a little.** Build the smallest thing that shows the idea works: a prototype on a few public documents, a notebook, or a clickable mock. Test it on at least 10 examples you create, and report honestly where it fails. This entire step is optional!

Think hard about the architecture part

We put real weight on architectural thinking. We are not looking for the most complex system; we want a design that fits the problem in a way others might not have thought of, and reasoning that holds up when we challenge it. Some questions worth asking yourself:

- Is one LLM call enough, or does the problem need retrieval over source documents, a rules engine, a deterministic calculator, several cooperating agents, or a mix? Why that combination?
- Which parts must be exact and auditable (a tax amount, a cited clause) and which can be probabilistic? How do you keep them apart?
- Where does the knowledge live: in the model, in a retrieval index, in code or in a human? What happens when the knowledge is no longer valid?
- Can every output be traced back to its source, so an auditor or regulator can check it?
- What is the weakest point of your design, and what would break it?
- What did you consider and reject, and why?

A simple design with sharp reasoning beats an elaborate one you can't defend.

Show what you added beyond the AI

We expect you to use LLMs to research, brainstorm and build; so will everyone else. What we want to see is the thinking the AI didn't do for you. Add a one-page appendix to your brief that covers:

- **Where AI helped:** the main things you used it for, with two or three of your key prompts.
- **What you rejected or corrected:** suggestions that were wrong, generic or unsafe for this domain, and how you knew.
- **What came from you:** the decisions, insights or design choices the AI did not suggest, such as something you learned from a real document or practitioner, a trade-off you made, or a risk you spotted.
- **One thing you'd stand behind without AI:** the part of your solution you understand well enough to defend live with no tools.

SECTION 3 - SUBMISSION

A form would be circulated around the following items:

1. **Written brief (4 pages max).** Please try to include:
    - the area you explored and what surprised you;
    - your problem statement: who, what task, how often, what it costs today;

- What you added beyond the AI
- your solution design, with an architecture diagram showing where the human and the AI each act, the alternatives you rejected and why;
- risks: wrong answers, outdated rules, confidential data, liability, and how your design handles each;
- Things you would consider before spending more time around this problem.

2. **Prototype or mock.** A link to a repo, notebook or clickable mock, with a README explaining how to run it and the examples you tested. This part is optional. **If a link is shared , please make sure the link would be publicly accessible.**
3. **LLM logs.** Show us the conversion you had with the LLM. A zip file of the chats can be uploaded in the form.

GUIDELINES

- **Where do I get the data?** Find it yourself. Public sources work well: published tax treaties, government tax guides, regulator circulars, public company annual reports, sample contracts. Make up realistic examples where you need them, and say which ones are made up.
- **Can I use real client or personal financial data?** No. Use only public or synthetic data.
- **Do I need finance experience?** No. We are testing how fast you learn a domain and how well you ask questions. Show your learning in the brief.
- **Which country's rules should I use?** Any. Say which jurisdiction your solution covers and what would change for another.
- **How much code do you expect?** As little as proves the point. A clickable mock with a clear explanation is acceptable if your strength is design and discovery; a working prototype helps if it's quick for you.
- **Do I need to pay for an AI model?** No. You can create a free account on OpenRouter and get an API key for its free-tier models. Free models have rate limits, so keep your tests small, and note in your README which model you used.
- **Is giving tax or legal advice a problem?** It is a real constraint, so treat it as part of the design. Explain who is accountable for the output and how your solution keeps a qualified human in the loop where the law requires one.